



UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

TRACE-BASED SYNC-PRESERVING GENERALIZED DEADLOCK PREDICTION

Absolvent
Lucan Cristian

Coordonator științific
Prof. Dr. Paul Irofti

București, iunie 2026

Abstract

Deadlocks are notorious heisenbugs that occur in multithreaded programs when two or more threads enter a circular wait state due to resource contention. Recent work [1] has made significant progress toward efficient and sound deadlock prediction on program traces that contain only lock operations. To achieve that, they defined sync-preserving deadlocks, a new class of deadlocks predictable in linear time with respect to the number of events in the trace. Other approaches [2] have prioritized completeness by addressing the often-overlooked deadlocks that arise from condition variable misuse, generalizing the concept of lock dependency to treat both condition variables and locks as resources that can be held and requested.

Our paper synthesizes these methodologies, combining and extending their core concepts and introducing a new property for the definition of correct trace reorderings, which reduces the number of false positives. As a result, we provide an open-source implementation that is more sound, complete, and efficient, achieving a **65% runtime reduction** and **81% peak memory usage reduction** compared to previous work [1]. The artifact of this paper can be found here <https://github.com/lucan8/GSPDOOffline>.

Keywords: Dynamic Program Analysis, Concurrency, Deadlocks, Predictive Analysis, Condition Variables

Rezumat

Deadlock-urile sunt bug-uri notorii și greu de reprodus, care pot apărea atunci când două sau mai multe fire de execuție ale unui program intră într-o stare de așteptare circulară, cauzată de lupta pentru resurse. Un studiu recent [1] a făcut progrese semnificative către o predicție mai eficientă și mai corectă a deadlock-urilor, ce pot fi derivate folosind urmele de execuție, care contin numai operatii pe mutexi, ale unui program. Pentru a realiza acest lucru, autorii definesc o nouă clasă de deadlock-uri predictibile în timp liniar, în raport cu numărul de evenimente, numite deadlock-uri "sync-preserving". Alte abordări [2] au priorizat completitudinea prin adresarea deadlock-urilor care apar ca rezultat al folosirii incorecte a variabilelor condiționate, generalizând conceptul de "lock dependency" pentru a trata atât variabilele condiționate, cât și mutex-urile ca fiind resurse ce pot fi cerute și deținute.

Lucrarea noastră sintetizează aceste metodologii, combinând și extinzând conceptele lor principale și introducând o nouă proprietate pentru definirea reordonărilor corecte, care reduce numărul de fals-pozitive. Astfel, oferim o implementare de tip "open-source" care este mai corectă, mai completă și mai eficientă, aducând o **reducere a timpului de execuție cu 65% și a consumului maxim de memorie cu 81%** comparativ cu [1]. Artefactul acestui studiu poate fi găsit aici <https://github.com/lucan8/GSPDOOffline>.

Cuvinte Cheie: Analiza Dinamica a Programelor, Concurența, Deadlock-uri, Analiza Predictivă, Variabile Condiționate

Contents

1	Domain background	3
1.1	Preamble	3
1.2	Deadlock Detection and Prediction	4
1.3	Algorithms and Techniques for deadlock prediction	4
1.3.1	Core Ideas	4
1.3.2	The Contribution of SPDOffline [1]	6
1.3.3	The Contribution of Unhang [2]	8
2	Methodology	8
2.1	Definitions and Concepts	8
2.2	Algorithms	10
2.2.1	Event Handler	10
2.2.2	Cycle Enumerator and Checker	11
3	Implementation	12
3.1	Preprocessing	12
3.2	Event Handling and Execution	12
3.2.1	Vector Clocks	13
4	Results	15
5	Conclusions and Future Work	17
A	Detailed Vector Clock Comparison	20

1 Domain background

This section will start by defining deadlocks and condition variables, illustrating these concepts with two programs that exhibit distinct deadlock behaviors. After that, the differences between deadlock detection and prediction will be discussed, the advantages and disadvantages of each method, and different ways of implementing them. Finally, the concepts and algorithms used for deadlock prediction will be presented.

1.1 Preamble

A **deadlock** is defined by an event based circular wait between two or more threads [3]. The type of deadlock is defined by the type of events. The literature focuses mainly on two of them [2]: **resource deadlocks**, in which threads are waiting to acquire locks held by other threads (so the events here are acquire and release) and **communication deadlocks**, where threads are waiting to be signaled by other threads that cannot do so (so the events here are signal, wait and broadcast). The latter type can actually come from a combination of signal and lock based waits, but also because of lost signals. In this paper, both will be addressed, taking into consideration only conditional variables for the communication deadlocks, ignoring the problem of missed signal which will be addressed in the future.

A **condition variable** is a synchronization primitive that is usually defined by a sleep queue (the actual condition variable object) and its associated lock which protects access to shared variables that act as conditions. It has two main operations: **wait** and **notify**. **Wait** releases the associated lock and blocks. After being unblocked, the lock is automatically re-acquired. It is usually called after evaluating a condition and finding it to be false, meaning that the thread must wait for the shared state to change before it can proceed. **Notify** unblocks a blocked thread. It is usually called after setting a condition to true.

Pgm 1: Resource deadlock-prone program

```
1 Lock l1, l2
2
3 func f1(){
4     l1.acquire()
5     l2.acquire()
6     l2.release()
7     l1.release()
8 }
9
10 func f2(){
11     l2.acquire()
12     l1.acquire()
13     l1.release()
14     l2.release()
15 }
16
17 func main(){
18     Thread t1, t2
19     t1.execute(f1)
20     t2.execute(f2)
21 }
```

Pgm 2: Communication deadlock-prone program

```
1 Lock l1, l_cv
2 bool cond
3 CondVar cv(l_cv)
4
5 func f1(){
6     l1.acquire()
7     l_cv.acquire()
8     while (!cond){
9         cv.wait(l_cv)
10    }
11    l_cv.release()
12    l1.release()
13 }
14
15 func f2(){
16     l1.acquire()
17     l1.release()
18
19     l_cv.acquire()
20     cond = true
21     cv.notify()
22     l_cv.release()
23 }
24
25 func main(){
26     cond = false
27     Thread t1, t2
28     t1.execute(f1)
29     t2.execute(f2)
30 }
```

The definition and usage of condition variables comes mostly from [3]. The idiomatic usage of condition variables can be observed in [pgm 2](#), as there is a condition associated with the primitive, it is verified in a while loop (lines 7-10), the notify is called whilst holding the associated lock and is accompanied by a write that changes the state of the variable (lines 19-22). This usage is good practice, not a necessity and it's

”wrong use” might create false positives that otherwise would be identified (more on this in [section 1.3.2](#)).

In [pgm 1](#), a deadlock can occur if t_1 acquires l_1 , then gets context switched with t_2 , which acquires l_2 (lines 4, 11). From here, t_2 will block trying to acquire l_1 and t_1 will block trying to acquire l_2 (lines 12, 5). In [pgm 2](#), a deadlock can occur if t_1 acquires l_1 , acquires l_{cv} , checks cond to false and calls wait on cv (lines 6-9). This automatically switches to t_2 which will be blocked waiting to acquire l_1 (line 16).

1.2 Deadlock Detection and Prediction

Deadlock detection is defined by an algorithm that observes a running program and detects deadlocks as they happen. This technique is usually accompanied by some sort of prevention or recovery system that helps the system escape the deadlock state and/or prevent future similar deadlocks [4, 5]. This technique can be quite costly depending on how often the detector is called; if one wants to find the deadlock exactly when it is about to happen, the detection algorithm would need to be called on each request, but doing so would impose considerable runtime overhead [3]. Undead [5], reduces this overhead by using a monitor thread that periodically takes a snapshot of the current state of the system and checks it for deadlocks. The prevention and recovery systems also introduce problems. Recovery systems are difficult to implement correctly as they need a well-defined victim policy that does not leave the system in a corrupted state upon process termination or a rollback system, which can impose high overhead [3].

Deadlock prediction is defined by an algorithm that, given a program, predicts possible deadlocks. This can be done statically, by analyzing the source code [6, 7, 8]. This has the benefit of not having to actually run the program, which might offer better coverage than dynamic options, but comes with the cost of not being very scalable and having a lot of false positives. Another option is to do it at runtime by deriving deadlock patterns from deadlock free executions [5, 2]. This offers more scalability and fewer false positives, but will make the implementation language dependent (the predictor has to be implemented in the same language as the program being observed), which might not be desirable in some situations. To overcome that, both ideas could be combined, first running the program and printing relevant operations (such as thread creation and lock acquisitions) to a file (called the trace of the program) and then running the actual predictor on the trace in an offline fashion similar to [9, 10, 1]. Some even take their predictions and try to verify that they are actually real deadlocks using biased scheduling [11, 12].

My work focuses on deadlock prediction, combining the ideas of [1] for efficiency and soundness and [2] for more deadlock coverage.

1.3 Algorithms and Techniques for deadlock prediction

This section presents the core mechanism of deadlock prediction—the lock dependency—along with the various ways it can be used to detect deadlocks. The discussion begins with foundational algorithms and concepts, tracking how they are expanded by [1] and [2] to introduce sync-preserving deadlocks and generalized dependencies, respectively. Finally, the limitations of the presented approaches will be analyzed.

1.3.1 Core Ideas

The core of most deadlock predictors is the **lock dependency**, defined as a tuple $\langle t, l, L \rangle$, where t is the thread, l is the requested lock, and L is the set of locks held by t just before requesting l . A **cyclic lock dependency chain** is an ordered sequence of dependencies $\langle d_1, d_2, \dots, d_n \rangle$, where $d_i = \langle t_i, l_i, L_i \rangle$ meet the following conditions: $l_i \in L_{i+1}$ for all $1 \leq i < n$, which closes the circular wait since $l_n \in L_1$ (1); $t_i \neq t_j$ for all $i \neq j$ (different threads) (2); $l_i \neq l_j$ for all $i \neq j$ (different desired locks) (3); $L_i \cap L_j = \emptyset$ for all $i \neq j$ (no common locks) (4). For [pgm 1](#) we have the dependencies $d_1 = \langle t_1, l_1, \emptyset \rangle$, $d_2 = \langle t_1, l_2, \{l_1\} \rangle$, $d_3 = \langle t_2, l_2, \emptyset \rangle$, $d_4 = \langle t_2, l_1, \{l_2\} \rangle$, with the cyclic dependency chain $\langle d_2, d_4 \rangle$.

The lock dependency was first introduced by [11] through the iGoodLock algorithm and then extensively used by all mentioned deadlock predictors, each focusing on either optimizing it or covering more deadlocks. Undead [5] brought forth the idea of ignoring first level lock acquisition and single-threaded activity as they

can never produce deadlocks (for [pgm 1](#) d_1 and d_3 would be ignored). Unhang [2] generalized the idea of dependency to take into account wait/signal events that appeared due to conditional variable misuse.

All dependency based algorithms track unique dependencies and enumerate cyclic lock chains. There are multiple ways of doing so, [5] uses the iGoodLock algorithm which iteratively builds cycles of increasing length, first creating all cycles of size two, then using that to build all cycles of size three and so on. Others [2, 1] construct a **dependency graph**, where nodes are defined by lock dependencies and an edge from $d_1 = \langle t_1, l_1, L_1 \rangle$ to $d_2 = \langle t_2, l_2, L_2 \rangle$ exists if and only if $l_1 \in L_2, t_1 \neq t_2, l_1 \neq l_2, L_1 \cap L_2 = \emptyset$. Then, cycle enumeration algorithms are used on the lock dependency graph to list all cycles and check if they are cyclic lock chains based on the conditions above. Some stop here and call this a predictable deadlock [5, 2], but doing this can introduce false positives because control flow is not considered.

SPDOffline [1] solves the problem of false positives by introducing a new class of deadlocks, called sync preserving deadlocks, which requires tracking read/write operations on variables without reasoning about the actual values of the variables, nor about the condition checks in branch statements.

Pgm 3: Pseudocode for [1] true negative	Pgm 4: Pseudocode for possi- ble dead thread false positive	Pgm 5: Pseudocode for [1] cond var false positive
<pre> 1 Lock l1, l2 2 int x = 0 3 4 func f1(){ 5 l1.acquire() 6 l2.acquire() 7 x = 1 8 l2.release() 9 l1.release() 10 } 11 12 func f2(){ 13 l2.acquire() 14 if (x == 1){ 15 l1.acquire() 16 l1.release() 17 } 18 l2.release() 19 } 20 21 func main(){ 22 Thread t1, t2 23 t1.execute(f1) 24 t2.execute(f2) 25 }</pre>	<pre> 1 Lock l1, l2 2 func f1(){ 3 l1.acquire() 4 l2.acquire() 5 l2.release() 6 l1.release() 7 } 8 9 func f2(){ 10 l2.acquire() 11 l1.acquire() 12 l1.release() 13 l2.release() 14 } 15 16 func main(){ 17 Thread t1, t2 18 19 t1.execute(f1) 20 t1.join() 21 22 t2.execute(f2) 23 t2.join() 24 }</pre>	<pre> 1 Lock l1, l2, l_cv 2 CondVar cv(l_cv) 3 4 func f1(){ 5 l_cv.acquire() 6 cv.wait(l_cv) 7 l_cv.release() 8 9 l1.acquire() 10 l2.acquire() 11 l2.release() 12 l1.release() 13 } 14 15 func f2(){ 16 l2.acquire() 17 l1.acquire() 18 l1.release() 19 l2.release() 20 21 l_cv.acquire() 22 cv.notify() 23 l_cv.release() 24 } 25 26 func main(){ 27 Thread t1, t2 28 29 t1.execute(f1) 30 t2.execute(f2) 31 }</pre>

In [pgm 3](#) a deadlock can never occur, if t_1 acquires l_1 , t_2 acquires l_2 (lines 5, 13), t_1 will have to wait until l_2 is released by t_2 which checks the condition to false (line 14) and releases the lock (line 18), letting t_1 also finish its execution. In this case, both [5] and [1] will correctly say that there is no deadlock. But in the scenario in which t_1 executes all of its operations and then t_2 does the same, without interruption, notice that the same dependencies and cycles are created as for [pgm 1](#), which means that [5] will report a deadlock, unlike [1] which assumes an ordering between the reads and writes done on the same variable.

It is also worth mentioning that the algorithms discussed can work both in online and offline settings. Undead [5] works in an online fashion, instrumenting the program taken as input by inserting hooks on important operations like acquire/release and handling each event appropriately (ex: acquire would need to update the thread's current lockset and create a new dependency depending on context). Then, as mentioned before, it runs its predictor at program exits. However, its algorithm could be very easily adapted to handle events coming from a program trace rather than directly from the program being analyzed. The trace is usually produced by first instrumenting the program as earlier, but instead of consuming the events directly, the instrumented program simply prints the operation being hooked in a given format to an output file.

SPDOffline [1] and this paper’s algorithm implement offline predictors that run on such traces.

1.3.2 The Contribution of SPDOffline [1]

SPDOffline is an open source(as part of their paper’s artifact), sound, trace based deadlock predictor that runs in linear time (with respect to the number of events in the trace), implemented in Java. In this section, the concepts they introduced will be discussed as well as some of their limitations.

From now on, traces will be used to reason about programs and predict deadlocks. A **trace** is defined by a sequence of events, which are defined by tuples of the form $\langle id, t, op \rangle$, where id is the trace line number, t is the thread and op is the operation performed by the thread. For now, the possible operations are **acquire**, **release**, **read**, and **write** (denoted as acq , rel , r , and w), with the former two operating on locks and the latter two on variables. It is assumed that the trace is well-formed (e.g. threads don’t release the locks held by others, every acquire must be preceded by a release, except for the first one).

Tr 1: Possible execution trace for [pgm 3](#) with artificial extension

t_1	t_2	t_1	t_2
1	$acq(\ell_1)$	11	$acq(\ell_1)$
2	$acq(\ell_2)$	12	$acq(\ell_2)$
3	$w(x)$	13	$rel(\ell_2)$
4	$rel(\ell_2)$	14	$rel(\ell_1)$
5	$rel(\ell_1)$	15	$acq(\ell_2)$
6	$acq(\ell_2)$	16	$acq(\ell_1)$
7	$r(x)$	17	$rel(\ell_1)$
8	$acq(\ell_1)$	18	$rel(\ell_2)$
9	$rel(\ell_1)$		
10	$rel(\ell_2)$		

Tr 2: Possible execution trace for [pgm 1](#)

t_1	t_2
1	$acq(\ell_1)$
2	$acq(\ell_2)$
3	$rel(\ell_2)$
4	$rel(\ell_1)$
5	$acq(\ell_2)$
6	$acq(\ell_1)$
7	$rel(\ell_1)$
8	$rel(\ell_2)$

Ro 1: Wrong reordering for [tr 1](#)

t_1	t_2
1	$acq(\ell_1)$
2	$acq(\ell_2)$
3	$r(x)$

Ro 2: Correct reordering for [tr 2](#)

t_1	t_2
1	$acq(\ell_1)$
2	$acq(\ell_2)$

From [tr 1](#), the cyclic lock dependency chain $D = \langle d_1, d_2 \rangle$, with $d_1 = \langle t_1, l_2, \{l_1\} \rangle$ and $d_2 = \langle t_2, l_1, \{l_2\} \rangle$ can be derived. SPDOffline [1] renames cyclic lock chains to **deadlock patterns** and also gives them an event-based representation; in this situation, one of them would be $D^{ins} = \langle e_2, e_8 \rangle$ (the events at which the dependencies that are part of the cycle are created). Events that yield the same dependencies are grouped into **abstract dependencies**, which are then used to define **abstract deadlock patterns**. In [tr 1](#), the lock dependencies d_1 and d_2 become part of the abstract dependencies $d_1^{abs} = \langle t_1, l_2, \{l_1\}, \{e_2, e_{12}\} \rangle$ and $d_2^{abs} = \langle t_2, l_1, \{l_2\}, \{e_8, e_{16}\} \rangle$ and the lock dependency chain D becomes an instance of the abstract deadlock pattern $D^{abs} = \langle d_1^{abs}, d_2^{abs} \rangle$. This abstract deadlock pattern has 4 instances, $D_1^{ins} = \langle e_2, e_8 \rangle$, $D_2^{ins} = \langle e_{12}, e_8 \rangle$, $D_3^{ins} = \langle e_2, e_{16} \rangle$ and $D_4^{ins} = \langle e_{12}, e_{16} \rangle$ (so all combinations of the events each abstract dependency groups).

As mentioned before, the lock graph is built (in this case, renamed to **abstract lock dependency graph**), where each node is an abstract dependency (construction is similar to [section 1.3.1](#)). This abstract dependency based construction reduces the size of the graph, which makes cycles enumeration more efficient without losing the actual deadlock pattern instances. Similarly to [5], they enumerate all cycles (using Johnson’s algorithm [13]), filter out those that are not abstract deadlock patterns (which happens if one of conditions (2), (3) or (4) for cyclic lock dependency chains is broken), and search for instances that are sync-preserving deadlocks.

A **sync-preserving deadlock** is a deadlock pattern $D^{ins} = \langle e_{a_1}, e_{a_2}, \dots, e_{a_n} \rangle$ for which there exists a sync-preserving reordering of the trace in which all events in D^{ins} are omitted, but all events preceding them (with respect to thread ordering) are present in the reordering. A **sync-preserving reordering** is a correct reordering in which all lock operations on the same lock are ordered based on the initial trace order. A **correct reordering** is a reordering of the trace operations that abides to the following rules: The same thread’s order of operations must be preserved (e.g., in [tr 1](#), e_2 cannot come after e_1) (1); if one thread operation appears in the reordering, all operations preceding it must appear as well (e.g., in [tr 1](#), if e_3 appears, e_1 and e_2 appear as well) (2); for all read operations appearing in the reordering, their corresponding last write operation must also be present (3). The **last write** of a read on a variable is the write operation performed on the same variable that is closest to that read with respect to the trace ordering, but before it.

The final condition is the most important because it enforces the preservation of the values of the

variables, and consequently, the control flow of the program. This is the reason why, in the case of [pgm 3](#) (or better said, [tr 1](#)), the algorithm identifies the deadlock pattern but does not report a deadlock. Upon closer inspection, [ro 1](#) would be the reordering required to observe the deadlock pattern $D_1^{ins} = \langle e_2, e_8 \rangle$. The problem is that the read operation performed by the second thread occurs before the write operation, which invalidates property (3) of correct reorderings. A reordering in which the last write rule is not preserved cannot truly be derived from the same trace. In this situation, reading the variable x before the write occurs implies different control flow than in the original execution trace as the value of x would be 0 instead of 1, meaning the deadlock pattern would not even appear. The reordering in [ro 2](#) is correct and observes the deadlock pattern $D_5^{ins} = \langle e_2, e_6 \rangle$.

To efficiently determine whether a deadlock pattern is sync-preserving, [1] equates the problem to computing the sync-preserving closure of the thread predecessors of each event in the deadlock pattern, and then checking if any of those events are part of the resulting closure. A **sync-preserving closure** for a set of events consists of: all events that precede them with respect to thread ordering (including the starting set of events for which the closure is computed) (1); the corresponding last write events for each read event (2); for any two lock events e_i and e_j where the operation is $acq(l_k)$ and e_i happens before e_j in the trace, it also contains the corresponding $rel(l_k)$ event of e_i (3). Conditions (1) and (2) construct correct reorderings, while (3) enforces the property of sync-preservation.

For the deadlock pattern $D^{ins} = \langle e_2, e_8 \rangle$ in [tr 1](#), the algorithm computes the sync-preserving closure $D^{ins'} = \langle e_1, e_2, e_3, e_4, e_6, e_7 \rangle$ through the following steps: it begins with $D_1^{ins'} = \langle e_1, e_7 \rangle$, as these are the thread predecessors of e_2 and e_8 . Condition (1) expands it to $D_2^{ins'} = \langle e_1, e_6, e_7 \rangle$ and condition (2) adds e_3 , as it is the corresponding last write of e_6 . Because e_3 was added, condition (1) dictates that e_2 must also become part of the closure. Finally, condition (3) adds e_4 (the corresponding release of e_2) because both threads attempt to acquire l_2 (e_2 and e_6), and e_2 occurs before e_6 in the trace. Because e_2 became part of the closure $D^{ins'}$, the algorithm determines that the deadlock pattern D^{ins} is not a sync-preserving deadlock and it is correctly suppressed.

Closures are represented using **thread-read-from timestamps**, which map threads to the number of operations executed by each. For example, the closure $D^{ins'} = \langle e_1, e_2, e_3, e_4, e_6, e_7 \rangle$ (which will be referred to as S) would be represented by the timestamp TS with the following values for each thread: $TS^S(t_1) = 4$, $TS^S(t_2) = 2$. The normal operations that can be done on timestamps are **point-wise comparison and maximum** (which should be seen as merging two timestamps into a single one, keeping the maximum value associated with each thread). **The timestamp of an event** is similar to computing its closure, counting the number of operations done by the thread to which the event belongs and for all read events the associated writes, ignoring the matching releases the closures also added. The read events can possibly introduce the number of operations done by other threads until their write. Note that this is also recursive, similar to closures, one event preceding the thread's one might be a read which will force a write from another thread to appear, which will force all its preceding operations to be counted, out of which one might be a read again and so on. **The timestamp for a set of events** is the timestamp resulting from merging the individual timestamp of each event.

Now, similar to how closures were created and used, timestamps will be computed for each thread predecessor of the events in the deadlock pattern, merged and instead of checking if any of the events in the pattern are part of the closure, the timestamp of each event is checked if it is $\leq$ to the merged timestamp mentioned earlier, if at least one has that property the deadlock is not reported. From $D^{ins} = \langle e_2, e_8 \rangle$, $TS^{e_2}(t_1) = 2$, $TS^{e_2}(t_2) = 0$, and $TS^{e_8}(t_1) = 3$, $TS^{e_8}(t_2) = 3$ are derived. Notice that $TS^{e_2} \leq TS^S$, which means the deadlock pattern will not be reported. Note that closures also added the associated release of each acquire done on the same lock, this is done for timestamps too, but it is a separate step of the algorithm that is discussed in [section 2.2.2](#).

Limitations. There are some limitations to the work in [1], mostly regarding false negatives, but false positives as well. An example of a **false negative** is [pgm 2](#). Because the algorithm is unaware of wait/notify operations, no trace produced by that program will ever generate a deadlock pattern, let alone a sync-preserving deadlock. An example of **false positive** can arise from the use of condition variables, such as [pgm 5](#) because the algorithm does not explicitly take into consideration the happens before relationship

of wait and notify. While this is a real false positive, it occurs in contexts where condition variables are not used in a very correct way, as normally wait and notify are accompanied by repeated reads and at least a write respectively (section 1.1). In such cases, [1] will correctly protect itself from false positives as the reads and writes that accompany wait and notify create exactly the expected ordering.

1.3.3 The Contribution of Unhang [2]

Unhang is a closed source, deadlock predictor that is implemented as a dynamic library for C/C++ applications. Unlike [1] and this paper, they do not analyze traces but the program while it is in execution.

Their primary contribution is the idea of **generalized dependency**. This approach treats condition variables as resources that can be held and requested, where a ‘wait’ operation acts as a request and a ‘notify’ operation acts as holding a resource. For example, [pgm 2](#) would create the following generalized dependencies (ignoring the first-level lock acquisitions): $d_1 = \langle t_1, l_2, \{l_1\} \rangle$, $d_2 = \langle t_1, cv, \{l_1\} \rangle$, $d_3 = \langle t_2, l_1, \{cv\} \rangle$, and $d_4 = \langle t_2, l_2, \{cv\} \rangle$. Notice that ‘notify’ acts as holding a resource for events that precede it (the last 8 in their implementation, which they call **recent statuses**). Also, the ‘wait’ operation does not count its associated lock as a held resource, being automatically released before the thread goes to sleep.

Instead of looking for lock dependency cycles, Unhang enumerates generalized dependency cycles following the conditions described in section 1.3.1 (in the earlier example, the cycle would be $D = \langle d_2, d_3 \rangle$), with two modifications: dependencies that hold the same condition variables **are not in conflict**, as condition signals are not mutually exclusive (1); dependencies that request condition variables with the same associated lock **are in conflict** (2). Conflicting dependencies cannot be part of the same dependency cycle, while non-conflicting dependencies can.

They also define the concept of **spurious cycles** that they do not report as real deadlocks. They are not addressed directly in this paper. Their algorithm also handles **missed signals** by verifying that the associated lock is a recent status when ‘notify’ is called. As mentioned previously, missed signals are ignored in the scope of this paper.

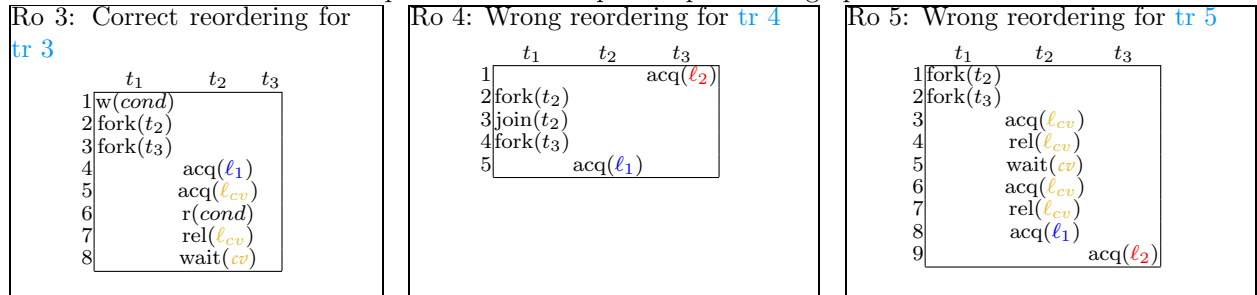
The contributions of [2] will help with both false positives and negatives resulting from the use of condition variables. Its limitations are similar to [5] and all algorithms that ignore control flow.

2 Methodology

As mentioned above, this paper combines the concepts introduced by [2, 1] and discussed in more detail in section 1.3.2 and section 1.3.3. This section begins by extending the concepts from section 1.3.2—most notably traces, deadlock patterns, and correct reorderings—to reduce the number of false positives and false negatives, before providing a detailed presentation of the actual algorithms.

2.1 Definitions and Concepts

Traces are extended to contain **fork**, **join**, **wait**, **notify**, and **notifyAll/broadcast**. The assumption of well-formedness still stands, with the important addition that the lock associated with a condition variable must be released before a wait operation and reacquired upon waking up.



Tr 3: Possible execution trace for
pgm 2

	t_1	t_2	t_3
1	w(<i>cond</i>)		
2	fork(t_2)		
3	fork(t_3)		
4			acq(ℓ_1)
5			rel(ℓ_1)
6		acq(ℓ_1)	
7		acq(ℓ_{cv})	
8		r(<i>cond</i>)	
9		rel(ℓ_{cv})	
10		wait(cv)	
11			acq(ℓ_{cv})
12			w(<i>cond</i>)
13			notify(cv)
14			rel(ℓ_{cv})
15		acq(ℓ_{cv})	
16		r(<i>cond</i>)	
17		rel(ℓ_{cv})	
18		rel(ℓ_1)	

Tr 4: Possible execution trace for
pgm 4

	t_1	t_2	t_3
1	fork(t_2)		
2		acq(ℓ_1)	
3		acq(ℓ_2)	
4		rel(ℓ_2)	
5		rel(ℓ_1)	
6	join(t_2)		
7	fork(t_3)		
8			acq(ℓ_2)
9			acq(ℓ_1)
10			rel(ℓ_1)
11			rel(ℓ_2)
12	join(t_3)		

Tr 5: Possible execution trace for
pgm 5

	t_1	t_2	t_3
1	fork(t_2)		
2	fork(t_3)		
3		acq(ℓ_{cv})	
4		rel(ℓ_{cv})	
5		wait(cv)	
6			acq(ℓ_2)
7			acq(ℓ_1)
8			rel(ℓ_1)
9			rel(ℓ_2)
10			acq(ℓ_{cv})
11			notify(cv)
12			rel(ℓ_{cv})
13		acq(ℓ_{cv})	
14		rel(ℓ_{cv})	
15		acq(ℓ_1)	
16		acq(ℓ_2)	
17		rel(ℓ_2)	
18		rel(ℓ_1)	

Deadlock patterns will now be based on generalized lock dependencies (defined in [section 1.3.3](#)) instead of normal lock dependencies as mentioned in [section 1.3.2](#). In the theoretical model, each thread has a starting lockset composed of all condition variables it has notified, but the implementation uses the last 8 operations. For [tr 3](#), the generalized abstract dependencies $d_1^{abs} = \langle t_2, cv, \{l_1\}, \{e_{10}\} \rangle$ and $d_2^{abs} = \langle t_3, l_1, \{cv\}, \{e_4\} \rangle$ create the abstract deadlock pattern $D_1^{abs} = \langle d_1^{abs}, d_2^{abs} \rangle$, which has as its only deadlock pattern instance $D_1^{ins} = \langle e_{10}, e_4 \rangle$.

Correct reorderings are extended to meet additional criteria: for every two threads t_1 and t_2 , where t_1 forks t_2 , the fork operation of t_1 must happen before the first operation of t_2 (4); for every two threads t_1 and t_2 , where t_2 joins with t_1 , the last operation of t_2 must happen before the join operation of t_1 (5); the successor (with respect to thread ordering) of every wait operation must happen after the notify that caused the wake-up (6), this captures the behavior of notifyAll as well, with the mention that it will have to happen before all wake-ups.

Properties (4) and (5) arise naturally, as forked threads have to acknowledge the operations done by the parent and parents have to acknowledge the operations done by the children when joining. Perhaps because they were evident, these properties were not clearly mentioned in the work of [1] even though their implementation enforces them. Reordering [ro 4](#) breaks property (4) because t_3 acquires lock l_2 (e_1) before t_1 forks it (e_3). Property (5) is broken because t_2 acquires l_1 (e_5) after joining with t_1 (e_3). If thread lifespans were ignored, this reordering would observe the deadlock pattern $D_2^{ins} = \langle e_3, e_9 \rangle$ and a deadlock would be reported (because it is also sync-preserving), which would create a false positive.

Property (6) tries to catch the essence of the happens-before relationship of wait and notify whilst balancing efficiency and restrictiveness. Normally, a notify can wake any of the threads that are sleeping, so creating a happens-before relationship only between the notifying and notified threads observed in the threads would ignore alternate executions in which another thread is chosen to wake up, but taking into account all such possibilities would be very inefficient and outside the scope of the predictor, it should perhaps be the job of the tool that produces the trace to either create multiple traces from the same program that differ in the threads chosen to wake up or create a bigger trace. Reordering [ro 5](#) breaks property (6) because t_2 reacquires the lock l_{cv} (the successor of the wait event e_5) without being woken up by t_3 (e_{11}). If the happens-before relationship between wait and notify operations was ignored, this reordering would observe the deadlock pattern $D_3^{ins} = \langle e_7, e_{16} \rangle$, which would also create a false positive. It is also important to notice that the thread-successor of any wait operation will be the acquisition of the associated lock. Reordering [ro 3](#) is correct and observes the deadlock pattern D_1^{ins} .

The definitions of **sync-preserving reordering**, **deadlock**, and **closure** stay the same, with the caveat that sync-preserving deadlocks use the updated definition of deadlock patterns. While not immediately obvious, the definition of sync-preservation does not change because the ordering of condition variables does not matter in the same way it does for locks, as condition variables do not provide mutual exclusion.

2.2 Algorithms

Given a well-formed trace, the algorithm reads each instruction and calls a handler based on the type of operation (from [algorithm 1](#)). At the end of trace parsing, the abstract dependency graph (`abs_dep_graph`) and the critical section history (`cs_hist`) are produced. Johnson's algorithm [13] is executed to enumerate all unique cycles in the graph. Subsequently, each cycle is evaluated; if it forms an abstract deadlock pattern, it proceeds to the final stage, which verifies whether it is sync-preserving using Algorithms 1 and 2 from [1].

2.2.1 Event Handler

Algorithm 1: Event Handler

```

1 cs_hist: keeps track of all critical sections, identified by a thread and a lock
2 abs_dep_graph: described in more detail in section 1.3.2
3 l_w: maps variables to the timestamp of the last write event
4 notif_q: maps condition variables to queues of notify events

// p and c come from parent and child
// ts comes from timestamp

FUNCTION OnFork( $t_p, t_c$ ):
5    $t_c.ts.MergeInto(t_p.ts)$ 

FUNCTION OnRead( $t, v$ ):
7    $t.ts.MergeInto(l_w[v])$ 

FUNCTION OnRequest( $t, l$ ):
9   if  $t.is\_asleep$  then
10      $notif\_ts := notif\_q[GetAssocCondVar(l)].Pop()$ 
11      $t.ts.MergeInto(notif\_ts)$ 
12      $t.is\_asleep := false$ 
13    $dep := abs\_dep\_graph.AddDep(t, l, t.lockset, t.ts)$ 
14    $t.recent\_status.Add(dep)$ 

FUNCTION OnJoin( $t_p, t_c$ ):
6    $t_p.ts.MergeInto(t_c.ts)$ 

FUNCTION OnWrite( $t, v$ ):
8    $l_w[v] := t.ts$ 

FUNCTION OnLock( $t, l$ ):
15   $t.lockset.Add(l)$ 
16   $cs\_hist.AddLock(t, l, t.ts)$ 

FUNCTION OnUnlock( $t, l$ ):
17   $t.lockset.Remove(l)$ 
18   $cs\_hist.AddUnlock(t, l, t.ts)$ 

FUNCTION OnNotify( $t, cv, is\_notify\_all$ ):
19   $new\_recent\_statuses := \emptyset$ 
20  for  $old\_dep \in t.recent\_status$  do
21    // Add cv to the lockset of old_dep
22     $new\_dep := abs\_dep\_graph.UpdateDep(old\_dep, cv)$ 
23     $new\_recent\_statuses.Add(new\_dep)$ 
24   $t.recent\_status := new\_recent\_statuses$ 
25   $notif\_q[cv].Push(t.ts, is\_notify\_all)$ 

FUNCTION OnWait( $t, cv$ ):
25   $dep := abs\_dep\_graph.AddDep(t, cv, t.lockset, t.ts)$ 
26   $t.recent\_status.Add(dep)$ 
27   $t.is\_asleep := true$ 

```

The `OnFork` and `OnJoin` (lines 5 and 6) handlers enforce the ordering given by properties (4) and (5) defined in [section 2.1](#), while `OnRead` and `OnWrite` (lines 7 and 8) enforce property (3) of correct reorderings, defined in [section 1.3.2](#). You might notice that, unlike the theoretical traces, in addition to the `OnLock` handler there is also `OnRequest`. This distinction is important because the time for a lock request might be different from the time of acquisition (when the thread blocks waiting) which might not even appear in the trace in the case of a deadlock happening.

The most important handlers are those that modify the output variables (`cs_hist` and `abs_dep_graph`, defined at lines 1 and 2), namely, the lock and condition variable operations. Ignoring the "sleepness" of the thread, `OnRequest` and `OnWait` are very similar; they create a generalized abstract dependency and add it to the graph and recent statuses (lines 13, 14 and 25, 26 respectively). `OnNotify` updates the dependencies found in the recent status of the threads by adding the condition variable to the lockset without changing the timestamp of events (lines 19-23). The `cs_hist` variable is modified only in the `OnLock` and `OnUnlock`

handlers (lines 16 and 18), as condition variable operations do not establish a critical section. This structure contains the timestamps of each acquire and release operation mapped to pairs of threads and locks. It is used when computing the sync-preserving closure to add the matching release of each acquire operation that is part of the reordering, except for the last one with respect to the trace ordering (following property (3) of sync-preserving closures). Coming back to thread "sleepness", three handlers are involved, `OnWait` just marks the threads as sleeping (line 27), `OnNotify` adds the timestamp of its event to the queue (line 24) and `OnRequest` merges the thread's timestamp with the timestamp at the top of the queue, representing property (6) defined in [section 2.1](#), and marks the thread as not sleeping (lines 9-12). This implementation of property (6) is an approximation, as the order in which threads that wake up consume the signals in the queue does not truly reflect the order in which they were woken up. To correctly implement this, the notify operation would also have to specify which thread woke up, though that might be both difficult and time consuming. All of these handlers advance the thread's timestamp (by incrementing `t.ts[t]`).

2.2.2 Cycle Enumerator and Checker

The enumerated cycles are then verified as abstract deadlock patterns using conditions (2), (3), and (4) from [section 1.3.1](#), with the addition of condition (1) from [section 1.3.3](#) (the second condition is omitted due to implementation limitations that are discussed in [section 3.2](#)).

Sync-Preservation Check (Algorithm 2 of [1]). Abstract deadlock patterns are checked for sync-preserving instances using Algorithm 2 from [1]. This algorithm incrementally creates combinations of events clustered by each abstract dependency (line 5), computes their thread predecessors (line 6), calculates the timestamp representing the sync-preserving closure (line 7), and verifies if any of the events in the pattern have a timestamp that is $\leq$ the closure's timestamp (lines 8 and 9, as discussed in [section 1.3.2](#), though the condition at line 8 might look a bit reversed). Due to the monotonic nature of closures, not all possible combinations of events are checked; at each iteration, only events with a timestamp $\not\leq$ than the current one are included (lines 10 and 11).

Recall from [section 1.3.2](#) that the first instance of D^{abs} was $D_1^{ins} = \langle e_2, e_8 \rangle$, which had timestamps $TS^{e_2}(t_1) = 2$, $TS^{e_2}(t_2) = 0$ and $TS^{e_8}(t_1) = 3$, $TS^{e_8}(t_2) = 3$, while the timestamp of the closure of its predecessors was $TS^S(t_1) = 4$, $TS^S(t_2) = 2$. When determining the next combination of events to check, notice that $TS^{e_2} \leq TS^S$, meaning e_2 is no longer an eligible candidate from the group given by the abstract dependency d_1^{abs} (which represented the events $\{e_2, e_{12}\}$). The next eligible candidate is e_{12} , with $TS^{e_{12}}(t_1) = 6$ and $TS^{e_{12}}(t_2) = 0$. Since $TS^{e_8} \not\leq TS^S$, e_8 remains the candidate for d_2^{abs} . Consequently, the deadlock pattern $D_3^{ins} = \langle e_2, e_{16} \rangle$ is skipped. In this scenario, it is the only pattern that can be skipped.

Sync-Preserving Closures (Algorithm 1 of [1]). The timestamps of the events grouped by abstract dependencies are partially constructed using the event handlers in [algorithm 1](#), taking into consideration all the rules of correct reorderings but ignoring the property of sync-preservation. To correctly compute the sync-preserving closure, after merging the predecessor timestamps of the events composing the deadlock pattern, Algorithm 1 from [1] is executed on this "half" closure. For each lock, using `cs_hist`, it identifies the last acquire event with a timestamp $\leq$ the closure's timestamp, one such event per thread (lines 6 and 7). For all such acquires—except the last one with respect to the trace ordering—the timestamp of the matching release is merged into the closure's timestamp (lines 8 and 9). This process repeats until an iteration yields no changes to the closure, at which point the final closure is returned (lines 10 and 11).

For the earlier example of $D_1^{ins} = \langle e_2, e_8 \rangle$, its starting TS^S (which is $TS^S(t_1) = 3$, $TS^S(t_2) = 2$) would be passed into Algorithm 1 of [1]. The algorithm first evaluates l_1 and finds e_1 to be the only acquire event occurring before the closure's timestamp, meaning no release is added. Next, it evaluates l_2 and finds events e_2 and e_6 . Because e_2 occurs before e_6 with respect to the trace ordering, only the timestamp of the matching release of e_2 is merged into the closure's timestamp, updating $TS^S(t_1)$ from 3 to 4.

3 Implementation

While the implementation closely follows the algorithms detailed in [section 2.2](#), certain structural decisions need further discussion. Before the event handler in [algorithm 1](#) is invoked, events must be read from the trace file and converted into a compatible format. The trace parser operates in two passes: one for preprocessing (filtering out unnecessary locks, events, and variables, as well as loading metadata) and one for the actual event handling.

3.1 Preprocessing

The trace file begins with **metadata** containing the total number of threads, locks, variables, and events. This information is used to pre-allocate fixed-size vectors that utilize respective ids (such as the thread id) as indices. This approach yields highly cache-friendly data structures compared to alternatives like `std::unordered_map`, while also preventing costly runtime re-allocations. Following the metadata, the events are recorded in an **8-byte binary format**. This binary format was also adopted by [\[1\]](#) (which is implemented in the Rapid framework [\[14\]](#)). Each event consists of a thread id (10 bits), an operation (4 bits), a target (34 bits), a source code location (15 bits), and a sign bit.

The preprocessing stage reads events in blocks and extracts only the necessary information; for example, the source code location is ignored at this stage. It then updates the internal structures used to determine which events, locks, and variables can be safely discarded. Similar to the approach taken in [\[1\]](#), variables and locks accessed by only a single thread are ignored alongside their corresponding events. Extending their work, our implementation recalculates the ids and counts of locks and variables. This reduces the memory footprint of the structures used in subsequent steps, but both memory and runtime are not highly affected by this change.

Another optimization attempt this paper had was to let the preprocessor filter out redundant write events that do not influence a subsequent read. Specifically, all consecutive write events to a variable before a read are ignored, except for the final one. This mostly had slight negative effects on both runtime and memory so it was dropped.

3.2 Event Handling and Execution

During the second pass, the filtered events are re-read and dispatched to the event handler, which invokes the appropriate algorithmic routines (see [section 2.2](#)). The following sections detail the core optimizations and data structures utilized during this phase and the rationale behind their design.

Similar to the works of [\[1, 5, 2\]](#), the **first level lock acquisitions** are ignored for all threads that do not call notify, as they cannot contribute to finding cycles, never-mind deadlocks. The events that are executed in a **single-threaded context** can also be avoided, with the mention that locksets are still updated for `OnAcquire` and `OnRelease`.

The **abstract dependency graph** stores its nodes in a `std::set`. While this may introduce more overhead than an unordered container, the decision was made because C++ lacks a built-in hasher for complex custom types. Though this will be optimized in future revisions, it does not currently act as a major performance bottleneck. For efficiency, neighbor list construction is deferred to the end of the second pass. A helper structure is used during event handling to map locks and condition variables to the dependencies that contain them in their locksets. Finally, the neighbor lists are sorted, as Johnson’s algorithm [\[13\]](#) for cycle enumeration requires strict node ordering. This sorting step introduced negligible overhead across the tested benchmarks.

Events are grouped based on pairs of abstract dependencies and source code locations. After being evaluated for the properties discussed in [section 1.3.2](#), the abstract deadlock pattern components are transformed. They transition from an abstract dependency format (thread id, desired resource, and lockset) into a similar triplet that replaces the lockset with the source code location. This representation provides higher utility to the end-user, as pinpointing the location of the deadlocking operations is generally more actionable than

simply listing the held locks. Displaying both the locksets and the locations where they were originally acquired would be ideal, but it falls outside the scope of this paper.

To construct dependencies during `OnNotify` events, each thread maintains a **history of recent statuses**. This is implemented as an LRU (Least Recently Used) cache backed by a doubly linked list. To maximize cache locality and avoid dynamic allocation fragmentation, the list nodes are stored in a fixed-size array and referenced via indices rather than pointers. As detailed in [section 1.3.3](#), only the last 8 statuses (each represented by a dependency) are tracked per thread.

Each thread’s **lockset** is implemented as a sorted `std::vector` of resource id and count pairs, accommodating the fact that locks in many benchmarks are reentrant. We explicitly avoided `std::unordered_map` for this structure. Maps require frequent dynamic allocations that degrade cache locality, incur significant per-node space overhead, and complicate the strict ordering required between abstract dependencies. Because locksets are typically small, a simple sorted vector provides superior performance.

The Condition Variable Trick. Currently, the analyzed traces are produced by java programs that only use the object’s synchronization primitives. This means that the instrumentor assigns the same id to both the condition variable and the lock, giving the opportunity of setting the condition variable’s id to be the inverse of the lock id (`cv_id = -lock_id`). While this provides more memory and time efficiency than mapping lock ids to condition variable ids and vice versa, it is more error prone than the latter approach. Using the cleaner mapping becomes a necessity if a lock can be used by multiple condition variables, which is possible by using `java.util.concurrent.locks`. Currently, the instrumentor used does not track such operations; this will be solved in the future. This limitation also means that property (2) of conflicting condition variables can be ignored as two condition variables cannot share locks ([section 1.3.3](#)).

Cycle Enumeration and Deadlock Checking. The implementation of both follows closely with the pseudocode of [\[13\]](#) and Algorithms 1 and 2 of [\[1\]](#). It is worth mentioning that this paper used a new data structure called a **lazy queue** to help with ignoring visited or unimportant nodes for cycle enumeration and to ignore events for sync-preserving checking without destroying the graph and the critical section history. A lazy queue is a wrapper over a sorted container (a vector or deque) that keeps a pointer to the first active element and instead of popping, it just moves the start pointer to the next element, resetting when desired (for `cs_hist` for example). The most important method of this container is `pop_until` which “pops” all elements until the first that is greater than the one specified, binary searching for it. This is exactly what’s needed for lines 6 and 7 from [\[1\]](#) Algorithm 1 to ignore events from `cs_hist` and for lines 10 and 11 of Algorithm 2 to create the optimization discussed in [section 2.2.2](#) that ignores combinations of events that can’t produce a deadlock.

3.2.1 Vector Clocks

One of the most crucial and widely used components of this algorithm is the **vector clock** (previously referred to as timestamps, see [section 1.3.2](#)). They support three primary operations: **point-wise comparison**, **merge**, and **increment**. For comparison, $VC_1 \leq VC_2$ holds if $VC_1[tid] \leq VC_2[tid]$ for every thread id (tid); $VC_1 < VC_2$ holds if $VC_1 \leq VC_2$ and there exists at least one tid where $VC_1[tid] < VC_2[tid]$. Merging takes the maximum value at each point across the vectors, and incrementing adds one to the event count of the owner thread.

Each thread owns a vector clock, which is either updated, snapshotted, or stored during handler execution. The number of instances created throughout program execution scales linearly with the number of events. Because the volume of events vastly exceeds the number of threads, variables, and locks, this scales to define the overall algorithmic complexity. Therefore, the vector clock implementation must be highly optimized for both execution time and memory footprint.

Dense Vector Clocks

In the work of [\[1\]](#), vector clocks are implemented as constant-size vectors equal to the total number of threads in the trace. These vectors use the thread id as an index to store the number of events each thread executed (defaulting to 0, stored in 8 bytes). This structure will be referred to as a **dense vector clock**.

This approach can be highly inefficient for traces with a large number of threads (some benchmarks contain 801 threads), as individual vector clocks may utilize only a few entries because not all threads establish write/read orderings for one another. **The number of steps** for both comparison and merging is constant, bounded by the number of threads (T). **The memory footprint** required to store a dense vector clock is $8 \cdot T$ bytes.

Sparse Vector Clocks

To mitigate both internal fragmentation and runtime overhead, this implementation initially utilized **sparse vector clocks**, which store only the active entries (denoted as T^a), the tids and event counts when the latter is non-0. At the beginning of its lifecycle, a sparse vector clock contains very few entries, growing dynamically over time.

The number of steps required for comparing or merging two sparse vector clocks (with active element counts T_1^a and T_2^a) is $(T_1^a + T_2^a)$. This equals the complexity of merging two sorted arrays, as sparse vector clocks are strictly initialized with a single pair (owner thread, 1) and remain sorted through continuous merge operations. **The memory footprint** of a sparse vector clock is determined by multiplying the number of active elements by the pair size. While both versions allocate 8 bytes for the event count, the sparse version must also store the thread id. As detailed in [section 3.1](#), binary events allocate 10 bits for thread ids, which is padded to 2 bytes (16 bits) in this implementation. Thus, the total memory required per sparse vector clock is $10 \cdot T^a$ bytes.

In order to have the underlying structure have the desired memory footprint, a vector of pairs of tid and event count cannot be used as the compiler would pad the thread id to be 8 bytes, losing 6 per entry. To avoid that, tids and values will have to use separate vectors. Both the vector of pairs and pair of vectors approaches were tried, giving similar runtimes, the latter having around **18% reduction in terms of memory** and **12% more runtime**, which probably comes from the fact that the number of re-allocations doubles compared to the former. Compared to dense vector clocks, the pair of arrays approach yields a decrease of **27% for runtime** and a **69% reduction for memory usage**.

Adaptive Vector Clocks

The runtime overhead of sparse vector clocks exceeds that of the dense version during merge/compare operations when $T_1^a + T_2^a > T$. Similarly, memory usage becomes suboptimal when $T^a \cdot 10 > T \cdot 8$, which is $T^a > 0.8 \cdot T$.

To overcome these limitations, this paper introduces **adaptive vector clocks**. These clocks initialize as sparse and dynamically transition to dense when $T^a \geq \frac{T}{2}$. While this threshold is not the strictly optimal inflection point for either time or memory, it facilitates efficient memory reuse, avoiding re-allocations and has desirable side effects that reduce some comparison operations to $O(1)$ (as will be discussed below). Both states share the identical underlying memory structure: a vector of 8-byte blocks. The distinction lies entirely in interpretation. In sparse mode, a stride of 2 is utilized to access coordinate pairs (`vec[i]` is the thread id, and `vec[i + 1]` is the event count), meaning the number of active elements is half the total vector size. The dense version reverts to a standard stride of 1, operating identically to the description in [section 3.2.1](#). It is important to notice that this representation uses 8 bytes to represent a thread id that would need only 2 (when the vector clock is in sparse mode). How the lost 6 bytes can be saved without trading in the ability to re-use the structure and introducing more runtime overhead remains unsolved in this paper.

The transition cost involves resetting a T -sized buffer, iterating through and migrating the sparse elements, and swapping the pointers between the buffer and the sparse vector. This requires $T + T^a$ operations but occurs at most once per vector clock lifecycle. **Merging a dense into a sparse vector** forces a dense transition, requiring $2 \cdot T^a + T$ steps alongside necessary memory allocations for a temporary copy of the sparse version and another potential one for assigning the values of the dense to the sparse. **Merging a sparse into a dense, and comparing**, requires exactly T^a operations. Conversely, comparing (both $<$, $\leq$) a dense vector clock with a sparse one is always false. This occurs because sparse vector clocks can contain at most $\frac{T}{2}$ active elements, whereas dense vector clocks maintain at least $\frac{T}{2} + 1$. Consequently, a dense vector

clock will always contain at least one element that the sparse vector lacks, which is equivalent to possessing a non-zero event count for a thread that holds a 0 value in the sparse vector. This is the "desirable side-effect" mentioned earlier.

Adaptive vector clocks give a **25% runtime reduction** and a **14% decrease in memory usage** as compared to the two array sparse version. In [appendix A](#) one can find a more detailed comparison between the vector clock implementations; the benchmarks used for comparison are the same as the ones in [table 1](#).

4 Results

The experiments were executed on a 64-bit Windows 11 machine equipped with an AMD Ryzen 5 5600 3.3 GHz processor (384 KB L1, 3 MB L2, and 16 MB L3 cache) and 16 GB of RAM. The tool presented in this paper is implemented in C++23, configured via the CMake build generator, and compiled using Clang with the `-O2` optimization flag within a MinGW environment. For comparison, we evaluate against SPDOffline [1], which is implemented in Java 8 but executed using Java 21.

Several benchmarks from standard suites were utilized for evaluation: IBM ConTest [15], Java Grande [16], DaCapo [17], and SIR [18]. The evaluation of these benchmarks used execution traces obtained directly from the SPDOffline artifact to compare implementation efficiency and to serve as a baseline for resource deadlock prediction capability. In addition to these standard suites, custom benchmarks were introduced to specifically evaluate prediction capabilities regarding condition variables. Some of these are hand-written (e.g., `cond_var_fp` and `cond_var_fp_pap`), whereas others were translated to Java from C++ source code (`mysql_50038` [19] and `mysql_68251` [20]), deduced from Java stack traces and source code (`batik_42268` [21]), or adapted from pseudocode. Specifically, `apache_42031` [22] was adapted from the pseudocode in [23] (slides 22–23), while the pseudocode for `memcached_1.5.19_new` and `mysql_5.6.9_new` was sourced from [2]. These additional programs were instrumented using the CalFuzzer tool [24]. Note that the numerical suffixes in the benchmark names correspond to their respective bug ids. Benchmarks designated with the "new" suffix were discovered by [2] and do not have publicly available bug reports; for these, the numbers denote the software version.

The column names N, T, V, L, and A/R denote event, thread, variable, lock, and acquire/request counts, respectively. Our predictor is called **GSPDOffline**, being a generalized version of **SPDOffline**. The predictors were compared based on the number of predicted deadlocks, execution time, and peak memory usage. Execution time is measured internally by both predictors. To determine memory usage, our predictor queries the operating system API to report peak memory (e.g., using `GetProcessMemoryInfo` on Windows, abstracted via a cross-platform function). For SPDOffline, the predictor is executed with the JVM `-Xlog` flag (`-Xlog:gc=info,gc+heap+exit=info:file=gc.log`) to log all memory-related actions; a subsequent post-processing step calculates the peak memory usage. Each benchmark was run for 10 iterations, and the mean of both runtime and peak memory usage was taken.

From [table 1](#), it can be observed that both predictors yield the **same results in terms of the number of reported deadlocks**, but exhibit considerable differences in both runtime and memory usage. Across the aggregated totals, our predictor **reduces runtime by 65%** and **peak memory usage by 81%**.

The primary drivers of these performance improvements are a more efficient implementation of vector clocks and a block-based event processing model, which avoids keeping all trace events in memory simultaneously. Additionally, the use of lazy queues likely contributed to runtime reduction during cycle enumeration and sync-preserving checks. For instance, a relatively small benchmark, `jigsaw`, yielded 172 cycles and 70 concrete patterns, resulting in a higher runtime of 1.65 seconds for SPDOffline. In contrast, `Sor` is a larger trace with more threads and lock events, but it contains zero abstract dependencies (meaning it generates no nodes or cycles), resulting in a runtime of only 0.76 seconds. This contrast strongly indicates that cycle enumeration and sync-preserving checking might introduce extra overhead. In our implementation, `jigsaw` executes in just 0.08 seconds. An anomaly for our predictor is `Sor`, which reports unexpectedly high memory usage for its trace size: 1.74 GB compared to SPDOffline's 205 MB. Through profiling, it was revealed that `cs_hist` consumes most of that memory. This is likely due to the fact that this benchmark has the second-highest number of acquire events (719K) and a relatively large number of threads (301). However, it

is still unclear where the exact difference in memory usage between the two predictors comes from.

Table 1: Resource Deadlock Prediction Comparison

Benchmark	N	T	V	L	A/R	GSPDOffline			SPDOffline		
						Dlk	Time(s)	Mem(MB)	Dlk	Time(s)	Mem(MB)
Deadlock	31	3	4	3	8	0	0.00	5.14	0	0.09	14.86
NotADeadlock	48	3	4	5	16	0	0.00	5.14	0	0.09	14.90
Picklock	54	3	6	6	20	1	0.00	5.15	1	0.10	15.23
Bensalem	55	4	5	5	22	1	0.00	5.16	1	0.10	15.22
Transfer	60	3	11	4	12	0	0.00	5.14	0	0.10	14.90
Test-Dimminux	61	3	9	7	24	2	0.00	5.16	2	0.10	15.23
StringBuffer	66	3	14	4	14	3	0.00	5.15	3	0.10	15.23
Test-Calfuzzer	146	5	16	6	48	1	0.00	5.17	1	0.10	15.23
DiningPhil	260	6	21	6	100	1	0.00	5.17	1	0.11	15.24
HashTable	310	3	5	3	174	2	0.00	5.17	2	0.12	15.22
Account	679	6	47	7	134	0	0.00	5.18	0	0.11	15.23
Log4j2	1K	4	334	11	33	1	0.00	5.16	1	0.11	15.23
Dbcp1	2K	3	768	5	50	2	0.00	5.16	2	0.12	15.28
Dbcp2	2K	3	592	10	50	0	0.00	5.16	0	0.12	15.31
Derby2	2K	3	1K	4	8	1	0.00	5.15	1	0.11	15.34
RayTracer	30K	5	5K	15	4	0	0.00	5.28	0	0.11	16.00
jigsaw	142K	21	7K	1K	47K	2	0.08	15.10	2	1.65	156.00
elevator	245K	5	727	52	47K	0	0.02	10.34	0	0.28	28.00
hedc	409K	7	109K	8	12	0	0.02	14.74	0	0.26	36.00
JDBCMySQL-1	442K	3	73K	11	8K	2	0.01	6.15	2	0.52	59.79
JDBCMySQL-2	442K	3	73K	11	2K	1	0.01	5.69	1	0.40	49.05
JDBCMySQL-3	442K	3	73K	12	8K	1	0.01	6.16	1	0.56	61.22
JDBCMySQL-4	442K	3	73K	14	8K	2	0.01	6.18	2	0.61	61.70
cache4j	775K	2	46K	20	28K	0	0.02	8.93	0	0.48	58.60
lusearch	2M	41	30K	4K	130	0	0.04	11.96	0	0.66	118.60
ArrayList	2M	801	120K	802	162K	9	0.17	40.36	9	2.35	1.05K
IdentityHashMap	2M	801	496K	802	161K	1	0.11	43.35	1	2.04	1.99K
Stack	2M	801	118K	1K	384K	9	0.31	82.90	9	3.61	1.40K
Sor	3M	301	2K	3	719K	0	1.22	1.74K	0	0.74	205.00
LinkedList	3M	801	289K	802	162K	9	0.21	50.64	9	2.87	1.59K
HashMap	3M	801	554K	802	169K	2	0.14	46.39	2	2.20	2.23K
WeakHashMap	3M	801	540K	802	169K	2	0.14	45.98	2	2.30	2.18K
Swing	3M	8	30K	739	185K	0	0.09	28.94	0	1.23	222.10
Vector	3M	3	15	4	800K	1	0.25	74.72	1	2.33	492.40
LinkedHashMap	4M	801	617K	802	169K	2	0.14	45.48	2	2.41	2.45K
montecarlo	7M	3	850K	3	4	0	0.30	42.69	0	1.16	447.57
TreeMap	9M	801	493K	802	169K	2	0.22	45.74	2	2.56	2.10K
hsqldb	19M	46	945K	403	334K	0	1.14	159.16	0	3.31	1.29K
sunflow	21M	16	2M	12	1K	0	1.23	279.80	0	2.45	1.15K
jspider	21M	11	4M	15	10K	0	1.04	390.83	0	2.73	1.61K
tradesoap	42M	236	2M	6K	116K	0	4.43	1.01K	0	9.85	4.32K
tradebeans	42M	236	2M	6K	116K	0	3.21	1.02K	0	10.10	4.26K
eclipse	64M	15	10M	4K	334K	0	3.09	944.07	0	13.36	3.93K
TestPerf	80M	50	599	9	196K	0	3.20	25.33	0	9.43	1.68K
Groovy2	119M	13	12M	9K	21K	0	5.80	1.66K	0	12.28	4.50K
biojava	221M	6	121K	79	766	0	8.58	14.22	0	17.21	4.24K
graphchi	240M	20	24M	61	688	0	13.14	2.02K	0	27.16	8.86K
Total	929M	7K	66M	42K	4M	60	48.37	9.97K	60	140.77	53.06K

One might notice that the trace metrics in [table 1](#) differ from those presented by [\[1\]](#). First, `Tsp` is omitted from our evaluation because the trace available in the public artifact utilizes 2,000 threads (compared to the 6 reported in their paper), which exceeds the maximum thread cap of 1,023 present in both predictors. Second, the benchmark `lusearch` is significantly smaller in our evaluation, spanning 2M events compared to the 203M events reported in [\[1\]](#).

Table 2: Communication Deadlock Prediction Comparison

Benchmark	N	T	V	L	A/R	GSPDOffline			SPDOffline		
						Dlk	Time(s)	Mem(MB)	Dlk	Time(s)	Mem(MB)
batik_42268_dlf	33	3	2	4	10	1	0.00	5.17	0	0.08	14.42
apache_42031_dlf	35	3	2	4	10	1	0.00	5.17	0	0.08	14.43
cond_var_fp_pap_dlf	42	3	3	5	14	0	0.00	5.17	1	0.10	14.90
cond_var_fp_dlf	44	3	3	5	14	0	0.00	5.18	1	0.10	14.91
mysql_68251_dlf	46	4	3	6	14	1	0.00	5.18	0	0.08	14.42
mysql_5.6.9_new_dlf	46	4	3	6	14	1	0.00	5.18	0	0.08	14.42
cond_var_fp_pap_idiomantic_dlf	46	3	4	5	14	0	0.00	5.17	0	0.10	15.03
cond_var_fp_idiomantic_dlf	48	3	4	5	14	0	0.00	5.18	0	0.10	15.11
mysql_50038_dlf	50	3	3	4	14	1	0.00	5.18	0	0.08	14.43
memcached_567_dlf	67	3	4	4	18	1	0.00	5.17	0	0.08	14.42
Total	457	32	31	48	136	6	0.00	51.76	2	0.88	146.48

The results presented in [table 2](#) demonstrate that the predictor introduced in this paper reduces both false positives and false negatives compared to SPDOffline. While SPDOffline is proven to be sound over traces that exclude condition variables, it fails when processing `wait` and `notify` operations. The benchmarks that yield false positives under SPDOffline are `cond_var_fp` and `cond_var_fp_pap`; the latter represents the program shown in [pgm 5](#), while the former is a closely related variant. Notably, their idiomatic counterparts, `cond_var_fp_idiomantic` and `cond_var_fp_pap_idiomantic`, are correctly handled as true negatives by both predictors. The remaining benchmarks represent real-world, documented bugs that exhibit distinct deadlock patterns; our predictor successfully identifies all of them, whereas SPDOffline misses them entirely.

5 Conclusions and Future Work

In this paper, we presented a time- and memory-efficient approach for deadlock prediction in execution traces containing both lock and condition variable operations. This was achieved by combining the works of [\[1\]](#) and [\[2\]](#) and extending them by introducing a new property (6 of correct reorderings) to avoid false positives from the use of condition variables. We also discussed and provided more efficient implementations for vector clocks that reduced both runtime and peak memory usage significantly.

Future work will expand upon this foundation in several key directions. First, we plan to extend the methodology to detect deadlocks arising from the misuse of other synchronization primitives, such as semaphores and barriers. Second, because our predictor is fundamentally bounded by the control-flow paths captured in a given trace, we intend to integrate advanced trace generation techniques. Exploring coverage-guided fuzzing and concolic execution will allow us to systematically exercise diverse execution paths, thereby maximizing the predictor’s detection capabilities. Furthermore, we aim to improve the tool’s diagnostic output; the current reporting system—which relies on simple log files detailing thread ids, lock ids, and source locations—will be upgraded to provide more comprehensive and intuitive bug visualization. Finally, we plan to evaluate the predictor against a broader suite of large-scale, computationally intensive benchmarks that heavily utilize condition variables and against multiple other deadlock predictors.

References

- [1] H. C. Tunç, U. Mathur, A. Pavlogiannis, and M. Viswanathan, “Sound dynamic deadlock prediction in linear time,” *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, Jun. 2023. [Online]. Available: <https://doi.org/10.1145/3591291>
- [2] J. Zhou, H. Yang, J. Lange, and T. Liu, “Deadlock prediction via generalized dependency,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSSTA 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 455–466. [Online]. Available: <https://doi.org/10.1145/3533767.3534377>
- [3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.
- [4] H. Julia, D. M. Tralamazza, C. Zamfir, G. Candea *et al.*, “Deadlock immunity: Enabling systems to defend against deadlocks,” in *OSDI*, vol. 8, 2008, pp. 295–308.
- [5] J. Zhou, S. Silvestro, H. Liu, Y. Cai, and T. Liu, “Undead: Detecting and preventing deadlocks in production software,” in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017, pp. 729–740.
- [6] D. Engler and K. Ashcraft, “Racerx: effective, static detection of race conditions and deadlocks,” in *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles*, ser. SOSP ’03. New York, NY, USA: Association for Computing Machinery, 2003, p. 237–252. [Online]. Available: <https://doi.org/10.1145/945445.945468>
- [7] J. Brotherston, P. Brunet, N. Gorogiannis, and M. Kanovich, “A compositional deadlock detector for android java,” in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2021, pp. 955–966.
- [8] Y. Cai, P. Yao, C. Ye, and C. Zhang, “Place your locks well: Understanding and detecting lock misuse bugs,” in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 3727–3744. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/cai-yuandao>
- [9] S. Bensalem and K. Havelund, “Dynamic deadlock analysis of multi-threaded programs,” in *Proceedings of the First Haifa International Conference on Hardware and Software Verification and Testing*, ser. HVC’05. Berlin, Heidelberg: Springer-Verlag, 2005, p. 208–223. [Online]. Available: https://doi.org/10.1007/11678779_15
- [10] C. G. Kalhauge and J. Palsberg, “Sound deadlock prediction,” *Proc. ACM Program. Lang.*, vol. 2, no. OOPSLA, Oct. 2018. [Online]. Available: <https://doi.org/10.1145/3276516>
- [11] P. Joshi, C.-S. Park, K. Sen, and M. Naik, “A randomized dynamic program analysis technique for detecting real deadlocks,” in *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI ’09. New York, NY, USA: Association for Computing Machinery, 2009, p. 110–120. [Online]. Available: <https://doi.org/10.1145/1542476.1542489>
- [12] Y. Cai and W. K. Chan, “Magicfuzzer: scalable deadlock detection for large-scale applications,” in *Proceedings of the 34th International Conference on Software Engineering*, ser. ICSE ’12. IEEE Press, 2012, p. 606–616.
- [13] D. B. Johnson, “Finding all the elementary circuits of a directed graph,” *SIAM Journal on Computing*, vol. 4, no. 1, pp. 77–84, 1975.

- [14] U. Mathur, A. Pavlogiannis, and M. Viswanathan, “Optimal prediction of synchronization-preserving races,” *Proc. ACM Program. Lang.*, vol. 5, no. POPL, Jan. 2021. [Online]. Available: <https://doi.org/10.1145/3434317>
- [15] E. Farchi, Y. Nir, and S. Ur, “Concurrent bug patterns and how to test them,” in *Proceedings International Parallel and Distributed Processing Symposium*, 2003, pp. 7 pp.–.
- [16] L. A. Smith, J. M. Bull, and J. Obdržálek, “A parallel java grande benchmark suite,” in *Proceedings of the 2001 ACM/IEEE Conference on Supercomputing*, ser. SC '01. New York, NY, USA: Association for Computing Machinery, 2001, p. 8. [Online]. Available: <https://doi.org/10.1145/582034.582042>
- [17] S. M. Blackburn, R. Garner, C. Hoffmann, A. M. Khang, K. S. McKinley, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, M. Hirzel, A. Hosking, M. Jump, H. Lee, J. E. B. Moss, A. Phansalkar, D. Stefanović, T. VanDrunen, D. von Dincklage, and B. Wiedermann, “The dacapo benchmarks: java benchmarking development and analysis,” in *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications*, ser. OOPSLA '06. New York, NY, USA: Association for Computing Machinery, 2006, p. 169–190. [Online]. Available: <https://doi.org/10.1145/1167473.1167488>
- [18] H. Do, S. Elbaum, and G. Rothermel, “Supporting controlled experimentation with testing techniques: An infrastructure and its potential impact,” *Empirical Softw. Engg.*, vol. 10, no. 4, p. 405–435, Oct. 2005. [Online]. Available: <https://doi.org/10.1007/s10664-005-3861-2>
- [19] (2010) Mysql communication deadlock, bug id 50038: Deadlock on flush logs with concurrent dml and rbr. [Online]. Available: <https://bugs.mysql.com/bug.php?id=50038>
- [20] (2013) Mysql communication deadlock, bug id 68251: 3-way deadlock on semisync replication and binlog rotate. [Online]. Available: <https://bugs.mysql.com/bug.php?id=68251>
- [21] (2007) Batik communication deadlock, bug id 42268: Deadlock when jsvgcomponent.stopprocessing() invoked from swing thread. [Online]. Available: https://bz.apache.org/bugzilla/show_bug.cgi?id=42268
- [22] (2010) Apache communication deadlock, bug id 42031: Eventmpm child process freeze. [Online]. Available: https://bz.apache.org/bugzilla/show_bug.cgi?id=42031
- [23] P. University. (2010) Deadlocks (cos 318: Operating systems). [Online]. Available: <https://www.cs.princeton.edu/courses/archive/fall10/cos318/lectures/Deadlocks>
- [24] P. Joshi, M. Naik, C.-S. Park, and K. Sen, “Calfuzzer: An extensible active testing framework for concurrent programs,” in *Proceedings of the 21st International Conference on Computer Aided Verification*, ser. CAV '09. Berlin, Heidelberg: Springer-Verlag, 2009, p. 675–681. [Online]. Available: https://doi.org/10.1007/978-3-642-02658-4_54

Appendix A Detailed Vector Clock Comparison

Table 3: Benchmark Evaluation comparing GSPDOffline Vector Clock (VC) Implementations

Benchmark	N	T	V	L	A/R	Adaptive-VC			Sparse-VC			Dense-VC		
						Dlk	Time(s)	Mem(MB)	Dlk	Time(s)	Mem(MB)	Dlk	Time(s)	Mem(MB)
Deadlock	31	3	4	3	8	0	0.00	5.13	0	0.00	5.13	0	0.00	5.13
NotADeadlock	48	3	4	5	16	0	0.00	5.13	0	0.00	5.13	0	0.00	5.13
Picklock	54	3	6	6	20	1	0.00	5.14	1	0.00	5.14	1	0.00	5.13
Bensalem	55	4	5	5	22	1	0.00	5.14	1	0.00	5.14	1	0.00	5.13
Transfer	60	3	11	4	12	0	0.00	5.13	0	0.00	5.13	0	0.00	5.13
Test-Dimminux	61	3	9	7	24	2	0.00	5.13	2	0.00	5.14	2	0.00	5.13
StringBuffer	66	3	14	4	14	3	0.00	5.13	3	0.00	5.13	3	0.00	5.13
Test-CalFuzzer	146	5	16	6	48	1	0.00	5.15	1	0.00	5.15	1	0.00	5.14
DiningPhil	260	6	21	6	100	1	0.00	5.16	1	0.00	5.16	1	0.00	5.15
HashTable	310	3	5	3	174	2	0.00	5.15	2	0.00	5.16	2	0.00	5.14
Account	679	6	47	7	134	0	0.00	5.16	0	0.00	5.17	0	0.00	5.16
Log4j2	1K	4	334	11	33	1	0.00	5.14	1	0.00	5.15	1	0.00	5.14
Dhcp1	2K	3	768	5	50	2	0.00	5.15	2	0.00	5.15	2	0.00	5.14
Dhcp2	2K	3	592	10	50	0	0.00	5.14	0	0.00	5.15	0	0.00	5.14
Derby2	2K	3	1K	4	8	1	0.00	5.14	1	0.00	5.14	1	0.00	5.13
RayTracer	30K	5	5K	15	4	0	0.00	5.26	0	0.00	5.31	0	0.00	5.30
jigsaw	142K	21	7K	1K	47K	2	0.08	15.09	2	0.08	15.39	2	0.09	17.95
elevator	245K	5	727	52	47K	0	0.02	10.34	0	0.02	12.05	0	0.02	10.40
hedc	409K	7	109K	8	12	0	0.02	14.72	0	0.03	15.76	0	0.02	15.60
JDBCMySQL-1	442K	3	73K	11	8K	2	0.01	6.11	2	0.01	6.39	2	0.01	6.14
JDBCMySQL-2	442K	3	73K	11	2K	1	0.01	5.67	1	0.01	5.78	1	0.01	5.67
JDBCMySQL-3	442K	3	73K	12	8K	1	0.01	6.18	1	0.01	6.31	1	0.01	6.23
JDBCMySQL-4	442K	3	73K	14	8K	2	0.01	6.12	2	0.01	6.39	2	0.01	6.22
cache4j	775K	2	46K	20	28K	0	0.02	8.91	0	0.03	10.87	0	0.02	9.03
lusearch	2M	41	30K	4K	130	0	0.04	11.95	0	0.04	10.97	0	0.04	16.14
ArrayList	2M	801	120K	802	162K	9	0.17	40.36	9	0.19	46.22	9	2.33	1.39K
IdentityHashMap	2M	801	496K	802	161K	1	1.11	43.34	1	0.13	49.35	1	1.23	1.39K
Stack	2M	801	118K	1K	384K	9	0.31	82.89	9	0.35	97.15	9	3.68	3.21K
Sor	3M	301	2K	3	719K	0	1.21	1.74K	0	3.39	2.20K	0	1.81	2.16K
LinkedList	3M	801	289K	802	162K	9	0.20	50.64	9	0.23	59.48	9	3.07	2.20K
HashMap	3M	801	554K	802	169K	2	0.14	46.38	2	0.16	53.68	2	1.69	1.71K
WeakHashMap	3M	801	540K	802	169K	2	0.14	45.98	2	0.16	52.96	2	1.66	1.64K
Swing	3M	8	30K	739	185K	0	0.10	29.01	0	0.11	30.95	0	0.10	32.96
Vector	3M	3	15	4	800K	1	0.25	74.73	1	0.40	112.89	1	0.28	74.73
LinkedHashMap	4M	801	617K	802	169K	2	0.14	45.45	2	0.16	52.03	2	1.61	1.53K
montecarlo	7M	3	850K	3	4	0	0.30	42.68	0	0.41	64.22	0	0.29	43.13
TreeMap	9M	801	493K	802	169K	2	0.22	45.76	2	0.24	52.42	2	2.29	1.58K
hsqldb	19M	46	945K	403	334K	0	1.15	159.25	0	1.36	185.93	0	1.87	578.49
sunflow	21M	16	2M	12	1K	0	1.21	279.78	0	1.74	293.57	0	1.15	323.74
jspider	21M	11	4M	15	10K	0	1.02	390.84	0	1.47	516.62	0	1.47	611.52
tradesoap	42M	236	2M	6K	116K	0	4.42	1.01K	0	5.86	968.65	0	11.70	5.62K
tradebeans	42M	236	2M	6K	116K	0	3.24	1.02K	0	6.71	1.02K	0	11.62	5.62K
eclipse	64M	15	10M	4K	334K	0	3.10	944.08	0	5.25	1.18K	0	4.14	1.58K
TestPerf	80M	50	599	9	196K	0	3.17	25.32	0	3.32	31.22	0	5.27	94.69
Groovy2	119M	13	12M	9K	21K	0	5.83	1.66K	0	8.67	1.77K	0	6.34	1.81K
biojava	221M	6	121K	79	766	0	8.63	14.20	0	8.85	17.01	0	7.84	16.90
graphchi	240M	20	24M	61	688	0	13.55	2.02K	0	15.77	2.61K	0	18.45	5.04K
Total	929M	7K	66M	42K	4M	60	48.83	9.97K	60	65.16	11.64K	60	90.12	38.43K